Efficient transit network design and frequencies setting multi-objective optimization by Alternating Objective Genetic Algorithm

Renato Oliveira Arbex, Claudio Barbieri da Cunha

Problem statement / 1

Definition

The *Transit Network Design and Frequency Setting Problem* (**TNDFSP**)

- requires to compute a set of Service Lines (*routes* + *trip frequencies*)
- given the Infrastructure Network and its associated Demand Matrix.

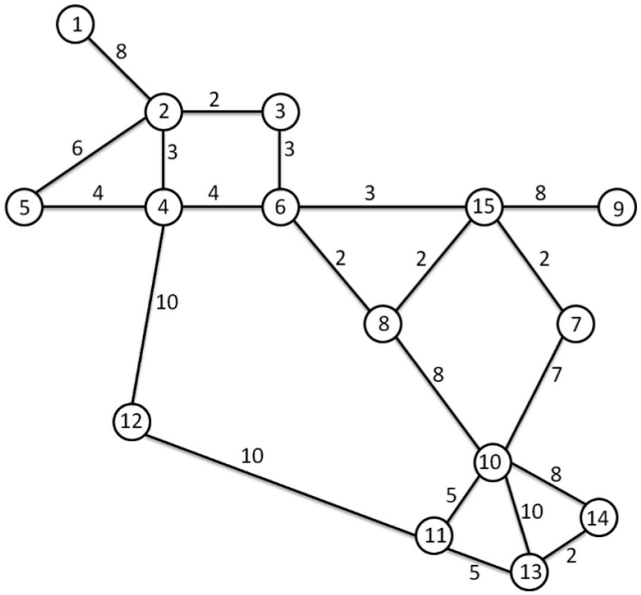


Figure: Example of INPUT Network

Origin/ Destination	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	0	400	200	60	80	150	75	75	30	160	30	25	35	0	0
2	400	0	50	120	20	180	90	90	15	130	20	10	10	5	0
3	200	50	0	40	60	180	90	90	15	45	20	10	10	5	0
4	60	120	40	0	50	100	50	50	15	240	40	25	10	5	0
5	80	20	60	50	0	50	25	25	10	120	20	15	5	0	0
6	150	180	180	100	50	0	100	100	30	880	60	15	15	10	0
7	75	90	90	50	25	100	0	50	15	440	35	10	10	5	0
8	75	90	90	50	25	100	50	0	15	440	35	10	10	5	0
9	30	15	15	15	10	30	15	15	0	140	20	5	0	0	0
10	160	130	45	240	120	880	440	440	140	0	600	250	500	200	0
11	30	20	20	40	20	60	35	35	20	600	0	75	95	15	0
12	25	10	10	25	15	15	10	10	5	250	75	0	70	0	0
13	35	10	10	10	5	15	10	10	0	500	95	70	0	45	0
14	0	5	5	5	0	10	5	5	0	200	15	0	45	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure: Example of INPUT Demand

Problem statement / 2

Definition

The *Transit Network Design and Frequency Setting Problem* (**TNDFSP**)

- requires to compute a set of Service Lines (*routes* + *trip frequencies*)
- given the Infrastructure Network and its associated Demand Matrix.

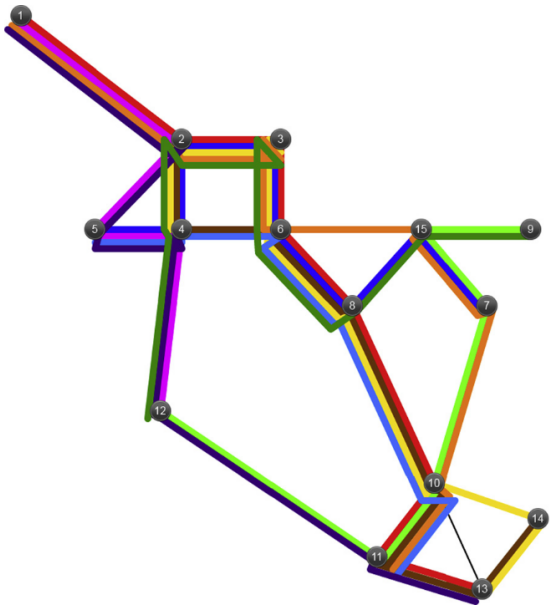


Figure: Example of OUTPUT Lines

Problem statement / 3

Objective Functions

$$\begin{aligned} min\ F_1 &= C_{wt} \times TWT + TIVIT + \Sigma_{\alpha=1}^{2} C_{\alpha t} \times T\alpha T + C_{ud} \times UD \\ min\ F_2 &= \Sigma_{s \in S} VN_s \end{aligned} \tag{1}$$

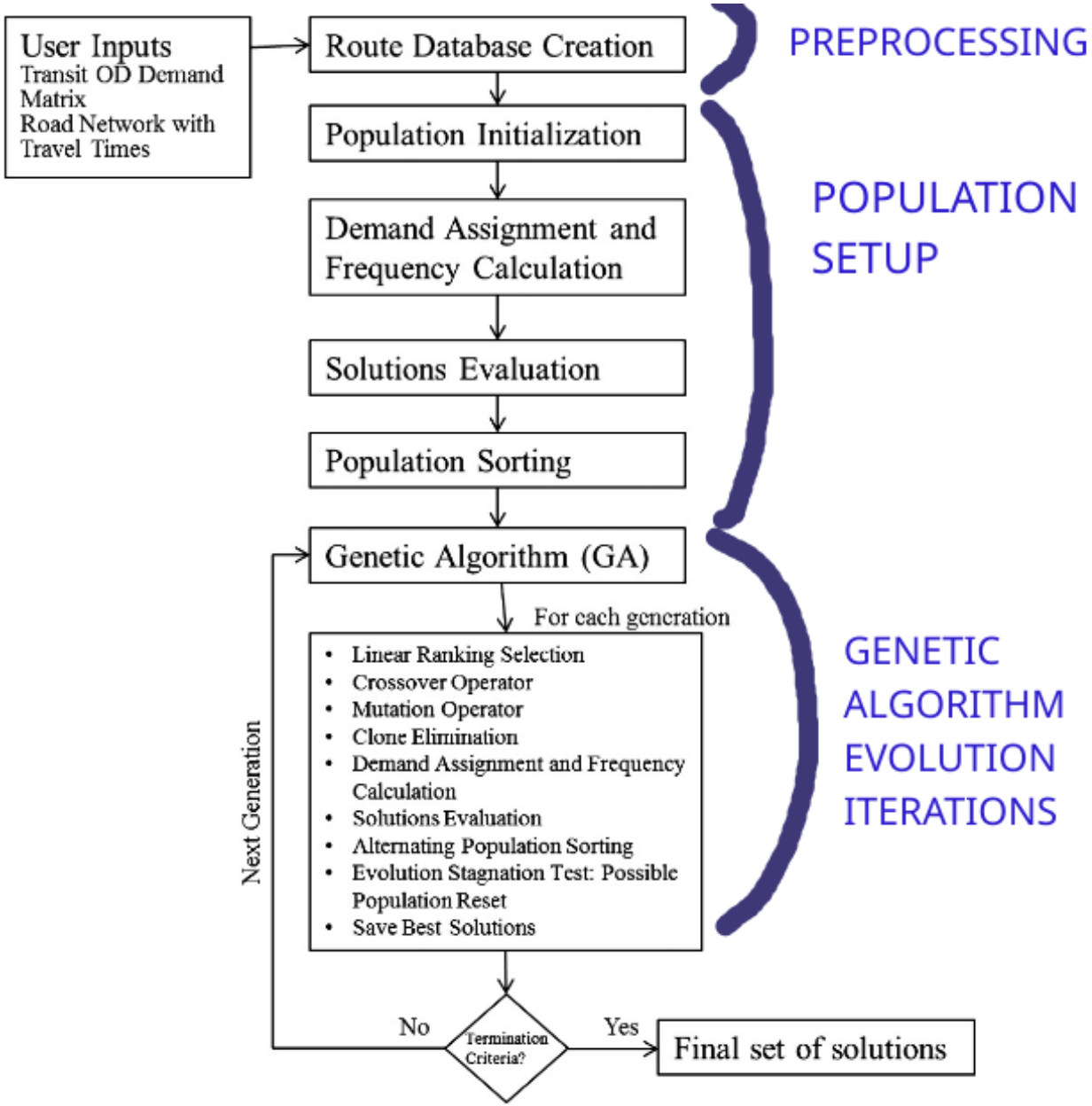
- TWT total waiting time for all passengers
- C_{wt} waiting time weight
- $TIVTT$ total in-vehicle travel time for all passengers
- $T\alpha T$ total number of α-*th* transfers required by all passengers
- $C_{\alpha t}$ α-*th* transfer penalty
- UD unsatisfied demand
- C_{ud} penalty for unsatisfied demand
- VC_s fleet size of one service line

Problem statement / 4

Constraints

- A solution is a set of unique Service Lines forming a connected component covering the entire input network.
- A route is a path (of min length $= 3$) from a node A to a node B without repetitions of nodes.
- Each bus has 40 seats and cannot hold more than 50 passengers.

A Genetic Algorithm for TNDFSP



Route Database Creation / 1

Definition

- The Route DB is a collection of paths.
- Each path is contained exactly once and has an associated ID.
- The Route DB is indexed so that it is possible to retrieve in $O(1)$ all the paths starting with node v, ending with node w or containing the edge (v, w)

Route Database Creation / 2

Route DB Construction

- For each pair of nodes $(v, w) \in V \times V$ such that $OD[v, w] \neq 0$, add the *k-shortest-paths* from v to w.
- k is not fixed but is expressed as *detour ratio* $R \in (0, 1)$.
- Calling C_p the cost of the path p and C_o the minimal cost of a shortest path, it selects all paths p such that $\frac{C_p - C_o}{C_o} \leq R$.
- The *k-shortest-paths* for a pair v, w are obtained using the **Yen** Algorithm, which in turn uses the **Dijkstra** algorithm on increasingly modified versions of the adjacency matrix of the graph.

Route Database Creation / 3

Construction Time Complexity

- The Dijkstra Algorithm is $O(|E| + |V| log|V|)$
- The Yen Algorithm calls Dijkstra $k|V|$ times
- Assuming verifying path membership is $O(1)$.
- Assuming adding a path to the DB is $O(1)$.
- Constructing Route DB is $O(k|V|^3(|E| + |V| log|V|))$

Construction Space Complexity

- The Dijkstra Algorithm is $\Theta(|V|)$
- The Yen Algorithm produces k paths with $\Theta(k|V|)$ space
- Constructing Route DB is $O(k|V|^3)$

Population Initialization / 1

Individual's Generation

- An *isolated node* has only 1 edge connecting it to the network (e.g. nodes 1, 9)
- For each *isolated node* v, add a random route starting/ending with v
- Track down which nodes are *served* by the current solution
- Until all nodes are served: randomly select a pair of unserved nodes v, w such that $OD[v, w] \leq 0$
- Add a random route containing the edge v, w
- Everytime check if the route is already in the solution (then select another one)
- If not all nodes can be served, the individual is discarded and a new one is generated instead

Population Initialization / 2

Individual's Feasibility Criteria

- Each node of the graph must be served and the individual shall form a connected component
- Each route shall contain at most one occurence of a node
- Each individual shall contain at most one occurrence of each route

Example of Individual

- R_1 : 1-2-3-6-8-10-11-13
- R_2 : 9-15-7-10-11-12
- R_3 : 7-15-8-6-3-2-4-5
- R_4 : 2-4-6-8-10-11-13-14
- R_5 : 13-14-10-8-6-3-2-4
- R_6 : 1-2-5-4-12
- R_7 : 11-10-7-15-6-3-2-1
- R_8 : 5-4-6-8-10-11
- R_9 : 13-11-12-4-5-2-1
- R_{10} : 9-15-8-6-3-2-4-12

Population Initialization / 3

Individual's Fate

- Each individual does not contain the routes as values but their ID inside the Route DB
- Each crossover operation creates a new entry for the Route DB
- A crossover operation could invalidate the individual (*unfeasible*)
- An individual which is not *feasible* is replaced by a freshly generated one
- The individual is assigned with served demand and trip frequencies
- The individual is evaluated with respect to User Cost (F_1) and Operator Cost (F_2)

Demand Assignment and Frequency Setting / 1

For each pair v, w such that $OD[v, w] \neq 0$, compute the probabilities P_j of a user wanting to travel from v to w using the combination of routes j. Those probabilities can be aggregated on route-level to compute the peak load of the route Q_r^{max}, and then the frequency f_r.

Probabilities for One/Two Transfer Paths

$$P_j = \frac{e^{-U_j}}{\Sigma_{i \in Options} e^{-U_i}} \tag{2}$$

$$U_i = T_w \cdot C_{wt} + T_{iv} + \Sigma_{\alpha=1}^{2} X_{\alpha t} \times C_{\alpha t} \tag{3}$$

Probabilities for Direct Paths

$$P_j = \frac{f_j}{\Sigma_{i \in Options} f_j} \tag{4}$$

Demand Assignment and Frequency Setting / 2

Frequency-based Bound

$$f_r = \frac{Q_r^{max}}{LF_r \cdot CAP} \leq f_{r_{max}} \tag{5}$$

- The frequencies of routes can also be computed by rewriting the equation *(5)* into *(6)*.
- Thus, all frequencies become parameters (initially set to 1) and incremented iteratively until all coefficients LF_r become $\leq LF_{max}$ (set to 1.25 by authors).
- NB: Realistically, frequency shall relate to an integer number of trips during the hour/day period.

Load-Factor-based Bound

$$LF_r = \frac{Q_r^{max}}{CAP \cdot f_r} \leq LF_{max} \tag{6}$$

Solutions Evaluation

At this point the individual is a set of defined routes + the allocated frequencies (thus the fleet size). This is enough to compute the objective functions F_1 and F_2 on the individual using equations 1.

Example of Frequency Setting and Individual Evaluation (see 11)

Route	Q_{max}	f	ht	fl
R_1	519.1	10.51	5.71	12
R_2	392.02	8.42	7.12	9
R_3	279.65	6.78	8.85	4
R_4	441.57	9.4	6.38	9
R_5	392.32	9.0	6.66	8
R_6	124.08	3.72	16.12	3
R_7	710.97	13.0	4.62	13
R_8	599.38	11.9	5.04	9
R_9	157.14	4.27	14.05	6
R_{10}	142.6	4.34	13.82	4

Param	Value
C_{wt}	2.0
C_{1t}	30
C_{2t}	40
f_{max}	13
f_{min}	1
F_1	458152
F_2	77

(Alternating) Population Sorting and Linear Ranking Selection

- The population is sorted by one of the two objective functions (F_1 and F_2), alternatively
- If in iteration t it was sorted with F_1, in iteration $t+1$ it is sorted with F_2
- The top k individuals out of the sorted population are selected for breeding, the rest is discarded

Crossover Operator

- Individuals are selected for crossover with linear ranking
- A single point of crossover k is selected randomly
- Offsprings are created by exchanging their k-th route
- Satisfability of offspring is checked and in case of damage, a new crossover point k' is selected
- If no more offspring can be generated, a new pair of parents is selected ...
- ... until at most 80% of possible pairs has been processed

Mutation Operator

- An individual mutates by replacing one of its routes with another one from the Route DB
- The new route is selected so that is serves the same OD Pair that originally generated the route
- Again, if the individual cannot be mutated without having an invalid offspring, a new individual is selected for mutation

Clone Elimination, Stagnation Test and Population Reset

- If two individuals are (symmetrically) equal, one is discarded and replaced by a newly generated one
- (Test) Whenever the lowest fleet and user cost is the same after more than 200 generations ...
- (Reset) ... the entire population is renewed using the initial population procedure
- The best *k-th* individuals (regardless if they are offsprings or parents) are pushed to the next generation (*elitism*)

Thank You